Deploying Confiance Services to a single AWS EC2 (t3.medium)

One Ubuntu box runs everything: **Next.js** (frontend) + **FastAPI** (backend) + **PostgreSQL** (DB) + **Nginx** (reverse proxy + SSL). Domain: **confianceservices.in** (API on **api.confianceservices.in**).

> Replace every `CHANGE_ME…` with a real value. Generate secrets: `openssl rand -base64 32` (NEXTAUTH_SECRET) · `openssl rand -hex 24` (ADMIN_API_KEY / DB password). Use the **same** `ADMIN_API_KEY` in the frontend and backend.

---

0. Before you start

- Push this repo to GitHub (so you can `git clone` it on the server). Commit the `deploy/` folder too.
- Have your `.pem` key and your GoDaddy login ready.

1. Launch the instance ✅ (already done)

You launched a **t3.medium** named `confiance` in **us-east-1 (N. Virginia)** — that's fine (Mumbai would've been slightly lower latency for India, but no need to redo). Settings used: - Region: **us-east-1 (N. Virginia)** — keep ALL the AWS steps below in this same region - AMI **Ubuntu Server 24.04 LTS** · Type **t3.medium** · Key pair `confiance-key` (`.pem` downloaded) - Security group inbound: **22 (My IP)**, **80 (Anywhere)**, **443 (Anywhere)** — nothing else - Storage **30 GB gp3**

2. Static IP + DNS

1. EC2 → **Network & Security** → **Elastic IPs** (region must be **us-east-1**, same as the instance) → **Allocate Elastic IP address** → keep defaults (Amazon's pool) → **Allocate**.
2. Select the new IP → **Actions** ▾ → **Associate Elastic IP address** → Resource type **Instance** → choose `confiance` → **Associate**. The Instances list now shows it in the *Elastic IP* column. **Copy that IP** (this is your permanent address — the temporary public IP changes on stop/start).
3. GoDaddy → **confianceservices.in** → **DNS** → **Manage** → add three **A records** (TTL 600): - `@` → A → `<Elastic IP>` - `www` → A → `<Elastic IP>` - `api` → A → `<Elastic IP>`

3. Connect + base packages

```
chmod 400 confiance-key.pem
ssh -i confiance-key.pem ubuntu@YOUR_ELASTIC_IP
```

```
sudo apt-get update && sudo apt-get upgrade -y
# 2 GB swap (so builds/Postgres never run out of memory)
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
# Node 20, Python, Postgres, Nginx, git, pm2
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs python3-venv python3-pip postgresql nginx git
sudo npm install -g pm2
```

4. PostgreSQL

```
sudo -u postgres psql -c "CREATE USER confiance WITH PASSWORD 'CHANGE_ME_DB_PASS';"
sudo -u postgres psql -c "CREATE DATABASE confiance OWNER confiance;"
sudo -u postgres psql -d confiance -c "ALTER SCHEMA public OWNER TO confiance;"
```

5. Clone the code

```
cd ~ && git clone YOUR_GITHUB_REPO_URL cofiance
```

6. Backend (FastAPI)

```
cd ~/cofiance/backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp ~/cofiance/deploy/env/backend.env.example .env
nano .env          # set DB password + ADMIN_API_KEY (matches frontend)
python -m app.seed # ⚠️ run ONCE only — it wipes + seeds the DB
deactivate
```

7. Frontend (Next.js)

```
cd ~/cofiance/frontend
npm ci
cp ~/cofiance/deploy/env/frontend.env.production.example .env.production
nano .env.production   # set NEXTAUTH_SECRET, ADMIN_* (ADMIN_API_KEY matches backend)
npm run build          # env must be set before this
```

8. Start both apps (pm2)

```
pm2 start ~/cofiance/deploy/ecosystem.config.js
pm2 save
pm2 startup            # run the `sudo env … systemctl enable` line it prints
pm2 status             # both `web` and `api` should be "online"
```

9. Nginx

```
sudo cp ~/cofiance/deploy/nginx-confiance.conf /etc/nginx/sites-available/confiance
sudo ln -s /etc/nginx/sites-available/confiance /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Now `http://confianceservices.in` should load (once DNS has propagated).

10. HTTPS (free SSL)

```
sudo apt-get install -y certbot python3-certbot-nginx
sudo certbot --nginx -d confianceservices.in -d www.confianceservices.in -d api.confianceservices.in
# choose "redirect" → HTTP auto-redirects to HTTPS; certs auto-renew
```

11. Verify

- `https://confianceservices.in` loads with a lock icon.
- `https://api.confianceservices.in/clients` returns JSON.
- Apply + Contact forms submit; admin login works at `https://confianceservices.in/admin`.

12. Nightly DB backup (no RDS → back up yourself)

```
chmod +x ~/cofiance/deploy/backup-db.sh
(crontab -l 2>/dev/null; echo "0 2 * * * ~/cofiance/deploy/backup-db.sh") | crontab -
```

Redeploying after code changes

```
cd ~/cofiance && git pull
cd backend  && source .venv/bin/activate && pip install -r requirements.txt && deactivate && pm2 restart api
cd ../frontend && npm ci && npm run build && pm2 restart web
```

Never re-run `python -m app.seed` after go-live — it deletes submitted applicant data.

Handy commands

- `pm2 status` · `pm2 logs web` · `pm2 logs api` · `pm2 restart all`
- `sudo systemctl reload nginx` · `sudo nginx -t`
- DB shell: `psql -U confiance confiance`

Security checklist

- [ ] Changed default `ADMIN_PASSWORD`
- [ ] Random `NEXTAUTH_SECRET`
- [ ] Random `ADMIN_API_KEY` (identical in frontend + backend)
- [ ] Strong Postgres password
- [ ] SSH (port 22) limited to your IP in the security group
- [ ] Ports 3000 / 8000 / 5432 NOT open to the internet (Nginx/localhost only)